МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ

«БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №2

Специальность ПО-12

Выполнил

С. Э. Михновец,

студент группы ПО-12

Проверила

А. Д. Кулик,

преп. стаж. кафедры ИИТ,

«___k _____2026 г.

Брест 2026

Цель работы: научиться разрабатывать простейшие программы на языке программирования Python.

Задание 1. Задание 1

Реализовать простой класс

Требования к выполнению

• Реализовать пользовательский класс по варианту.

Для каждого класса

• Создать атрибуты (поля) классов

• Создать методы классов

• Добавить необходимые свойства и сеттеры (по необходимости)

• Переопределить магические методы __str__ и __eq__

1) Равнобедренный треугольник, заданный длинами сторон – Предусмотреть

возможность определения площади и периметра, а также логический метод,

определяющий существует или такой треугольник. Конструктор должен

позволять создавать объекты с начальной инициализацией. Переопределить

метод __eq__, выполняющий сравнение объектов данного типа.

Выполнение:

```
"""
Модуль для работы с равнобедренными треугольниками.
Предоставляет расчет площади, периметра и проверку существования.
"""

import math


class Triangle:
    """
    Класс, представляющий равнобедренный треугольник.
    """

    def __init__(self, side, base):
        """Инициализация сторон треугольника."""
```

```
        self.side = side
        self.base = base

    def is_exists(self):
        """Проверяет, существует ли треугольник с данными сторонами."""
        return self.side > 0 and self.base > 0 and (2 * self.side >
self.base)

    def calc_square(self):
        """Вычисляет площадь треугольника через высоту."""
        if not self.is_exists():
            return 0.0
        height = math.sqrt(self.side**2 - (self.base / 2) ** 2)
        area = (self.base * height) / 2
        return area

    def calc_perimetr(self):
        """Вычисляет периметр треугольника."""
        return self.side * 2 + self.base

    def __eq__(self, other):
        """Сравнивает два треугольника по их сторонам."""
        if not isinstance(other, Triangle):
            return False
        return self.side == other.side and self.base == other.base


def main():
    """Демонстрация работы класса Triangle."""
    tri1 = Triangle(5, 6)
    tri2 = Triangle(5, 6)
    tri3 = Triangle(10, 6)
    tri4 = Triangle(-1, 6)

    print(f"Треугольник 5, 5, 6 существует? {tri1.is_exists()}")
    print(f"Треугольник 10, 10, 6 существует? {tri3.is_exists()}")
    print(f"Треугольник -1, -1, 6 существует? {tri4.is_exists()}")
    print(f"Треугольник tri1 и tri2 равны? {tri1 == tri2}")
    print(f"Треугольник tri1 - периметр: {tri1.calc_perimetr()}")
    print(f"Треугольник tri1 - площадь: {tri1.calc_square()}")


if __name__ == "__main__":
    main()
```

Рисунки с результатами работы программы

```
Треугольник 5, 5, 6 существует? True
Треугольник 10, 10, 6 существует? True
Треугольник -1, -1, 6 существует? False
Треугольник tri1 и tri2 равны? True
Треугольник tri1 - периметр: 16
Треугольник tri1 - площадь: 12.0
```

Задание 2. Построить модель программной системы с применением отношений

(обобщения, агрегации, ассоциации, реализации) между классами. Задать

атрибуты и методы классов. Реализовать (если необходимо)

дополнительные классы. Продемонстрировать работу разработанной системы

1) Система Факультатив. Преподаватель объявляет запись на Курс. Студент записывается на Курс, обучается и по окончании Преподаватель выставляет Оценку, которая сохраняется в Архиве. Студентов, Преподавателей и Курсов

при обучении может быть несколько.

Выполнение:

```
"""
Модуль для работы с классами для демонстрации обобщения, агрегации и
ассоциации.
В Python обобщение реализуется через наследование, ассоциация и агрегация
— через
хранение ссылок на другие объекты, а реализация — через интерфейсы.
В системе реализованы роли Преподавателя, Студента и сущности Курса и
Архива.
"""

from abc import ABC, abstractmethod


class BaseUser(ABC):
    """
    Абстрактный класс (Интерфейс), демонстрирующий 'Реализацию'.
    """

    @abstractmethod
    def get_role(self):
        """Возвращает роль пользователя."""

    @abstractmethod
    def get_name(self):
        """Возвращает имя пользователя."""


class User(BaseUser):
    """Базовый класс для всех пользователей системы (Обобщение)."""

    def __init__(self, name: str):
        """Инициализация пользователя."""
        self.name = name

    def get_role(self):
        """Реализация метода получения роли."""
        return "Пользователь"

    def get_name(self):
        """Реализация метода получения имени."""
        return self.name

    def __str__(self):
        return f"{self.get_role()}: {self.name}"
```

```
class Teacher(User):
    """Преподаватель, который ведет курсы и выставляет оценки."""

    def get_role(self):
        """Переопределение роли."""
        return "Преподаватель"

    def announce_course(self, title: str):
        """Преподаватель создает (объявляет) курс."""
        print(f"Преподаватель {self.name} открыл запись на курс
'{title}'.")
        return Course(title, self)

    def set_grade(self, student, course, value: int, archive):
        """Преподаватель выставляет оценку и отправляет её в архив."""
        msg = (
            f"Преподаватель {self.name} выставил оценку {value} "
            f"студенту {student.name} за курс '{course.title}'."
        )
        print(msg)
        archive.save_record(student, course, value)


class Student(User):
    """Студент, который записывается на курсы и обучается."""

    def get_role(self):
        """Переопределение роли."""
        return "Студент"

    def enroll(self, course):
        """Студент записывается на курс."""
        course.add_student(self)
        print(f"Студент {self.name} записался на курс '{course.title}'.")

    def study(self):
        """Процесс обучения."""
        print(f"Студент {self.name} активно изучает материалы курса...")


class Course:
    """Курс, который связывает преподавателя и студентов (Ассоциация)."""

    def __init__(self, title: str, teacher: Teacher):
        """Инициализация курса."""
        self.title = title
        self.teacher = teacher
        self.students = []

    def add_student(self, student: Student):
        """Добавление студента в список курса (Агрегация)."""
        if student not in self.students:
            self.students.append(student)

    def get_student_count(self):
        """Возвращает количество записанных студентов."""
        return len(self.students)


class Archive:
    """Класс-хранилище для результатов обучения."""
```

```
    def __init__(self):
        """Инициализация списка истории."""
        self.history = []

    def save_record(self, student, course, grade):
        """Сохранение записи в архив."""
        record = {
            "student": student.get_name(),
            "course": course.title,
            "grade": grade
        }
        self.history.append(record)

    def show_all(self):
        """Вывод всех записей архива."""
        print("\n--- СОДЕРЖИМОЕ АРХИВА ---")
        for entry in self.history:
            output = (
                f"Студент: {entry['student']} | "
                f"Курс: {entry['course']} | "
                f"Оценка: {entry['grade']}"
            )
            print(output)


def main():
    """Основная функция программы."""
    prof = Teacher("Крощенко А. А.")
    archive = Archive()

    python_course = prof.announce_course("Основы Python")

    vodim = Student("Вадим")
    bob = Student("Боб")

    vodim.enroll(python_course)
    bob.enroll(python_course)

    vodim.study()
    bob.study()

    prof.set_grade(vodim, python_course, 5, archive)
    prof.set_grade(bob, python_course, 4, archive)

    count = python_course.get_student_count()
    print(f"Всего студентов на курсе: {count}")
    archive.show_all()


if __name__ == "__main__":
    main()
```

Рисунки с результатами работы программы

```
Преподаватель Крощенко А. А. открыл запись на курс 'Основы Python'.
Студент Вадим записался на курс 'Основы Python'.
Студент Боб записался на курс 'Основы Python'.
Студент Вадим активно изучает материалы курса...
Студент Боб активно изучает материалы курса...
Преподаватель Крощенко А. А. выставил оценку 5 студенту Вадим за курс 'Основы Python'.
Преподаватель Крощенко А. А. выставил оценку 4 студенту Боб за курс 'Основы Python'.
Всего студентов на курсе: 2

--- СОДЕРЖИМОЕ АРХИВА ---
Студент: Вадим | Курс: Основы Python | Оценка: 5
Студент: Боб | Курс: Основы Python | Оценка: 4
PS D:\SPP>
```

Вывод: научился разрабатывать простейшие программы на Python.